A SMALL THEME FOR MODERN SLIDES

Python for Odoo Developers

Language foundations mapped to Odoo engineering work

Weblearns Academy
weblearns@training.local
Odoo Full-Stack Developer Program
Lecture materials / July 2026

ROADMAP

Agenda

TOC

1 A theme for modern talks

How this deck is taught

AWESOME-STYLE FRAME

Number rails, angled title art, and section slides are built in.

METROPOLIS DEFAULTS

Progress bars, block styles, and fonts are set from theme options.

DENSE TEACHING

Two-column frames pack objectives, examples, and pitfalls.

Python 3 | OOP | venv | PEP 8

1 P01 – P05

Python for Odoo Developers · teaching block 1

P01 What Is a Programming Language and Interpreter

Objectives, key points, and lab target

Objectives

- Explain what a programming language gives to humans and machines.
- Describe how an interpreter executes Python source code.
- Distinguish source code, bytecode, runtime objects, and process state.
- Connect interpreted execution to rapid Odoo development.

Key points

- A programming language defines syntax, semantics, and a standard way to express computation.
- The Python interpreter reads code, builds objects, executes statements, and reports errors.
- Interactive execution is useful for experiments, but production code must be repeatable.
- The same source code can behave differently when the environment, imports, or data changes.
- Odoo developers need both language knowledge and framework knowledge.

Lab target

Create a file named interpreter_lab.py that prints three values and their types. Run it, change one value, run again, and explain what the interpreter did each time.

P01 What Is a Programming Language and Interpreter

Teaching notes

What Is a Programming Language and Interpreter is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding the role of language rules, runtime behavior, and interpretation helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is source code is parsed into instructions that create and manipulate objects while the interpreter manages execution. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is thinking that code is just text rather than instructions with runtime state and side effects, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, server actions, models, controllers, and scripts are all Python instructions executed by a long-running...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P01 What Is a Programming Language and Interpreter

Example and pitfalls

Observe Python executing statements

The source code creates a string object, binds it to a name, and passes it to functions. Seeing the type and length makes the invisible runtime state concrete.

```
message = "hello Odoo developer"
print(message)
print(type(message))
print(len(message))
```

Common mistakes

- Treating Python files as configuration rather than executable code.
- Assuming an interactive experiment automatically matches Odoo's runtime environment.
- Ignoring errors because the interpreter stops at the first failing statement.
- Forgetting that a long-running Odoo process may need a restart to load changed code.

P02 Running Python, First Program, and Python 2 vs Python 3

Objectives, key points, and lab target

Objectives

- Run Python from a file and from an interactive shell.
- Write a first program that accepts input and prints output.
- Explain why Python 3 is the standard for modern Odoo work.
- Recognize Python 2 compatibility problems in old code.

Key points

- python3 usually selects Python 3 on Linux; python may vary by system.
- sys.executable shows the exact interpreter used for the running process.
- Python 3 treats print as a function and text as Unicode strings.
- Python 2 is end of life and should not be used for new Odoo development.
- Virtual environments change which interpreter and packages a command uses.

Lab target

Write a first_program.py file that asks for a developer name and prints the Python executable, version, and a greeting. Run it with the interpreter from a virtual environment if available.

P02 Running Python, First Program, and Python 2 vs Python 3

Teaching notes

Running Python, First Program, and Python 2 vs Python 3 is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding executing Python consistently and recognizing version boundaries helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a command chooses a specific interpreter binary, and that interpreter defines available syntax and libraries. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is running code with the wrong interpreter and misdiagnosing syntax or dependency errors, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, modern Odoo versions use Python 3, while older community snippets may still contain Python 2...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P02 Running Python, First Program, and Python 2 vs Python 3

Example and pitfalls

Confirm interpreter identity

This program proves which Python is running before any dependency debugging begins. It is a simple but valuable habit on servers with system Python, virtual environments, and...

```
import sys

print(sys.version)
print(sys.executable)
print("Hello from Python 3")
```

Common mistakes

- Using python when the server expects python3.
- Copying Python 2 snippets with print statements or old exception syntax.
- Installing packages into one interpreter and running another.
- Skipping version checks during deployment troubleshooting.

A small interactive program

```
name = input("Module name: ").strip()
print(f"Preparing scaffold for {name}")
```

P03 Data Types Overview

Objectives, key points, and lab target

Objectives

- Name the common built-in Python data types.
- Explain mutable versus immutable objects.
- Use type inspection to understand unfamiliar values.
- Choose appropriate data structures for Odoo code.

Key points

- Core types include None, bool, int, float, str, list, tuple, dict, set, and custom classes.
- Mutable objects can change in place; immutable objects produce new values when transformed.
- type(value) and isinstance(value, type) answer different questions.
- Choosing the right type communicates intent to future maintainers.
- Framework APIs often expect a specific type, not merely a similar-looking value.

Lab target

Build a table of ten Python values with their type, whether they are mutable, and one valid operation. Include at least one value that would commonly appear in Odoo data.

P03 Data Types Overview

Teaching notes

Data Types Overview is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding the shape and behavior of runtime values helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is every value is an object with a type, identity, and operations that may or may not mutate the object. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is using a value as if it were another type and producing subtle framework errors, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, recordsets, dictionaries, lists, strings, dates, booleans, and numbers move constantly between models,...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P03 Data Types Overview

Example and pitfalls

Inspect common values

repr shows an unambiguous display of each value, and type reveals what operations are valid. This is a useful debugging pattern when values arrive from JSON, forms, or ORM calls.

```
values = [None, True, 42, 19.5, "odoo", ["sale"], ("dr

for value in values:
    print(repr(value), type(value).__name__)
```

Common mistakes

- Assuming a string that looks like a number behaves like a number.
- Mutating a list that is shared by another part of the program.
- Using type(value) == SomeType when isinstance would handle subclasses better.
- Returning None where a caller expects an empty list or dictionary.

P04 Numbers, Math, and Operator Precedence

Objectives, key points, and lab target

Objectives

- Use integer, float, and decimal-style thinking appropriately.
- Apply arithmetic operators and understand division behavior.
- Explain how operator precedence affects expressions.
- Write clear numeric code for business logic.

Key points

- int represents whole numbers of arbitrary size.
- float represents binary floating-point values and can surprise in decimal money calculations.
- / returns true division, // returns floor division, and % returns remainder.
- ** binds more tightly than multiplication and addition.
- Parentheses are better than relying on readers to remember precedence.

Lab target

Implement a small price calculation with subtotal, discount, and tax. Write two versions, one compact and one using named intermediate values, then compare readability.

P04 Numbers, Math, and Operator Precedence

Teaching notes

Numbers, Math, and Operator Precedence is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding numeric operations and expression grouping helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is operators combine operands according to precedence rules unless parentheses make the intended order explicit. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is encoding financial or quantity calculations with unclear grouping or inappropriate float assumptions, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, prices, taxes, quantities, discounts, and stock moves depend on careful numeric logic. This means a...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P04 Numbers, Math, and Operator Precedence

Example and pitfalls

Make precedence visible

Both expressions are syntactically valid, but they encode different business rules. Parentheses turn the pricing rule into visible documentation.

```
subtotal = 100
discount = 10
tax_rate = 0.15

wrong = subtotal - discount * (1 + tax_rate)
right = (subtotal - discount) * (1 + tax_rate)

print(wrong)
print(right)
```

Common mistakes

- Using floats for money without understanding rounding requirements.
- Relying on precedence in long business expressions.
- Confusing / with // when calculating batches or pages.
- Forgetting that negative numbers affect floor division behavior.

Division operators

```
quantity = 17
box_size = 5

print(quantity / box_size)
print(quantity // box_size)
print(quantity % box_size)
```

P05 Variables, Expressions, and Statements

Objectives, key points, and lab target

Objectives

- Define variables as name bindings to objects.
- Distinguish expressions that produce values from statements that perform actions.
- Write assignments that clarify business intent.
- Avoid misleading names and accidental rebinding.

Key points

- Assignment binds a name to an object; it does not always copy the object.
- An expression can be used where a value is needed.
- A statement changes control flow, binding, or program state.
- Good variable names explain domain meaning, not just type.
- Rebinding a name can be clear or confusing depending on scope and length.

Lab target

Write a short script that builds an order summary from named variables. Then refactor it so each variable name explains a business concept rather than a data type.

P05 Variables, Expressions, and Statements

Teaching notes

Variables, Expressions, and Statements is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding names, values, and executable instructions helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a variable name points to an object, expressions compute values, and statements tell the interpreter to do work. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is believing assignment copies every object or that names and values are the same thing, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, model methods often combine ORM recordsets, computed values, and side-effect statements such as writes....
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P05 Variables, Expressions, and Statements

Example and pitfalls

Name binding with mutable values

same_tags and tags point to the same list, while copy_tags points to a separate list. This distinction matters when preparing values for ORM create and write calls.

```
tags = ["vip", "renewal"]
same_tags = tags
copy_tags = list(tags)

same_tags.append("priority")

print(tags)
print(copy_tags)
```

Common mistakes

- Using names like data or temp for important business concepts.
- Assuming assignment makes a deep copy.
- Hiding database writes among many harmless expressions.
- Reusing one variable name for unrelated meanings in the same method.

2 P06 – P10

Python for Odoo Developers · teaching block 2

P06 Strings: Concatenation, Formatting, Indexes, and Immutability

Objectives, key points, and lab target

Objectives

- Create and combine strings safely.
- Use f-strings for readable formatting.
- Index and slice strings without mutating them.
- Handle text values carefully in Odoo fields and logs.

Key points

- Strings can be indexed and sliced like sequences.
- Immutability means assigning to a string index is not allowed.
- f-strings are usually the clearest formatting tool for local variables.
- join is preferable for combining many strings.
- strip, lower, and replace return new strings.

Lab target

Create formatted labels for three fake invoices using f-strings. Include amount formatting, then demonstrate that calling upper on a string returns a new value.

P06 Strings: Concatenation, Formatting, Indexes, and Immutability

Teaching notes

Strings: Concatenation, Formatting, Indexes, and Immutability is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding text values and immutable sequence behavior helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a string is an immutable sequence of characters, so operations produce new strings instead of changing the original. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is building unclear or unsafe text output with manual concatenation and hidden conversions, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, names, emails, XML IDs, domains, log lines, and translated labels are all text-heavy surfaces. This...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P06 Strings: Concatenation, Formatting, Indexes, and Immutability

Example and pitfalls

Format a user-facing label

The f-string keeps the formatting rule close to the values and makes decimal display explicit. The slice returns a new string; it does not change the original label.

```
partner = "Azure Interior"
amount = 1250.5
currency = "USD"

label = f"{partner}: {amount:,.2f} {currency}"
print(label)
print(label[0:14])
```

Common mistakes

- Concatenating strings and numbers without explicit conversion or formatting.
- Expecting string methods to modify the original value.
- Using slicing indexes without naming why the slice matters.
- Manually building SQL or HTML strings instead of using safe APIs.

Join many parts

```
parts = ["sale", "order", "confirmed"]
xml_id = ".".join(parts)
print(xml_id)
```

P07 Booleans and Type Conversion

Objectives, key points, and lab target

Objectives

- Use True and False values in decisions.
- Explain truthy and falsy values.
- Convert between common types explicitly.
- Avoid ambiguous conversions in business rules.

Key points

- False, None, zero, empty strings, and empty containers are falsy.
- Non-empty strings are truthy, even the string 'False'.
- bool, int, float, str, list, and dict perform conversions when valid.
- Explicit conversion makes validation errors visible.
- Use is None when testing for the absence marker specifically.

Lab target

Write a parser for a yes/no text value that accepts yes, no, true, false, 1, and 0. Make invalid input raise ValueError with a clear message.

P07 Booleans and Type Conversion

Teaching notes

Booleans and Type Conversion is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding truth values and explicit boundaries between types helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is Python asks objects for truth value in conditionals, while conversion functions build new values of requested types. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is treating empty values, strings, numbers, and None as interchangeable, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, forms and integrations often deliver strings that must become booleans, numbers, dates, or empty values...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P07 Booleans and Type Conversion

Example and pitfalls

Truthiness surprises

The non-empty strings are truthy even though humans may read them as false-like. External text input should be parsed with domain-specific rules instead of passed directly to bool.

```
samples = [False, None, 0, "", [], "False", "0"]

for sample in samples:
    print(repr(sample), bool(sample))
```

Common mistakes

- Using bool('False') and expecting False.
- Testing if value when None and zero mean different things.
- Converting user input without handling ValueError.
- Letting raw JSON strings reach ORM write values unchecked.

Validate conversion

```
raw_quantity = "12"
quantity = int(raw_quantity)

if quantity <= 0:
    raise ValueError("quantity must be positive")

print(quantity * 2)
```

P08 Lists: Slicing, Matrix Data, Methods, and Unpacking

Objectives, key points, and lab target

Objectives

- Create and modify lists safely.
- Use indexes, slices, and common methods.
- Represent matrix-like data with nested lists.
- Unpack list values clearly.

Key points

- append adds one item; extend adds items from an iterable.
- Slicing returns a new list, while slice assignment mutates the original list.
- Nested lists can represent rows, but each row must be a separate list.
- Unpacking assigns multiple names from a sequence in one statement.
- sort mutates a list; sorted returns a new sorted list.

Lab target

Build a list of invoice rows, slice the first three, sort a copy by amount, and unpack each row into named variables while printing a summary.

P08 Lists: Slicing, Matrix Data, Methods, and Unpacking

Teaching notes

Lists: Slicing, Matrix Data, Methods, and Unpacking is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding ordered mutable collections helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a list stores references in order and can be changed in place by index, slice, or method calls. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is mutating shared lists or confusing shallow copies with independent nested data, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, Odoo code often builds command lists, report rows, import buffers, and wizard selections. This means a...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P08 Lists: Slicing, Matrix Data, Methods, and Unpacking

Example and pitfalls

List methods and slicing

append and item assignment change the list in place, while the slice creates a new list. This distinction is important when one function should not modify a caller's list.

```
states = ["draft", "sent", "sale"]
states.append("done")
first_two = states[:2]
states[1] = "quoted"

print(states)
print(first_two)
```

Common mistakes

- Using one inner list repeatedly when building a matrix.
- Calling sort when the original order must be preserved.
- Appending a list when extend was intended.
- Changing a list passed into a function without documenting that side effect.

Unpack rows

```
rows = [
    ["S0001", 100.0],
    ["S0002", 250.0],
]

for order_name, total in rows:
    print(f"{order_name}: {total:.2f}")
```

P09 Dictionaries: Keys, Values, and Methods

Objectives, key points, and lab target

Objectives

- Use dictionaries for named data.
- Access keys safely with direct lookup and get.
- Iterate over keys, values, and items.
- Prepare dictionaries for ORM create and write operations.

Key points

- Dictionary keys must be unique and hashable.
- dict[key] raises KeyError when missing; dict.get(key) can provide a default.
- items returns key-value pairs for iteration.
- update merges values and overwrites existing keys.
- in checks keys, not values.

Lab target

Create a dictionary for a fake product with name, default_code, list_price, and active. Validate required keys and print each key-value pair in a readable format.

P09 Dictionaries: Keys, Values, and Methods

Teaching notes

Dictionaries: Keys, Values, and Methods is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding mapping keys to values helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a dictionary stores unique hashable keys, and each key identifies one current value. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is using loose dictionaries without knowing which keys are required or optional, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, Odoo create and write methods receive dictionaries whose keys must match field names and whose values...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P09 Dictionaries: Keys, Values, and Methods

Example and pitfalls

Build ORM values

The dictionary mirrors the field-value contract used by Odoo's create method. Checking required keys before calling the ORM produces clearer errors.

```
partner_vals = {
    "name": "Azure Interior",
    "email": "info@example.com",
    "customer_rank": 1,
}

required = ["name", "email"]
missing = [key for key in required if not partner_vals.get(key)]
print(missing)
print(partner_vals.items())
```

Common mistakes

- Using get for required keys and accidentally accepting missing data.
- Overwriting a key during update without noticing.
- Assuming dictionaries preserve a business order in older Python contexts.
- Passing unknown field names into ORM write values.

P10 Tuples

Objectives, key points, and lab target

Objectives

- Create tuples and understand their immutability.
- Use tuples for fixed-size grouped values.
- Unpack tuples in assignments and loops.
- Recognize tuple syntax in Odoo domains and commands.

Key points

- A one-item tuple needs a trailing comma.
- Tuples are immutable, but contained mutable objects can still change.
- Tuple unpacking documents expected position meanings.
- Tuples are hashable only if all contained values are hashable.
- Odoo domains commonly use three-item tuples such as ('field', '=', value).

Lab target

Build three domain tuples and unpack them in a loop. Add one malformed tuple and observe the error, then explain why early failure is useful.

P10 Tuples

Teaching notes

Tuples is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding fixed grouped values helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a tuple is an immutable ordered container, often used when position and size are part of the meaning. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is confusing tuple immutability with immutability of objects inside the tuple, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, domains, selection values, and relational command patterns use tuple-shaped data frequently. This means...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P10 Tuples

Example and pitfalls

Domain tuple shape

Each tuple has a fixed three-part meaning, and unpacking enforces that expectation. This mirrors a common Odoo domain pattern.

```
domain = [
    ("state", "=", "sale"),
    ("amount_total", ">", 1000),
]

for field, operator, value in domain:
    print(field, operator, value)
```

Common mistakes

- Writing ('draft') and thinking it is a tuple instead of a string.
- Trying to assign to a tuple element.
- Assuming mutable objects inside a tuple cannot change.
- Using positional tuples where a dictionary would be clearer.

3 P11 – P15

Python for Odoo Developers · teaching block 3

P11 Sets

Objectives, key points, and lab target

Objectives

- Create sets for unique values.
- Use membership and set operations.
- Choose sets over lists for uniqueness and fast lookup.
- Apply sets to validation and import cleanup tasks.

Key points

- Sets remove duplicates automatically.
- Membership checks in sets are efficient.
- Union, intersection, and difference express comparison logic clearly.
- Sets are unordered and should not drive display order directly.
- Only hashable values can be set members.

Lab target

Given two lists of module technical names, find modules in both lists, modules only in the first list, and the sorted union of all modules.

P11 Sets

Teaching notes

Sets is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding unordered unique collections helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a set stores unique hashable values and supports mathematical operations such as union and intersection. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is using lists for uniqueness checks and producing duplicate business actions, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, imports, access checks, tags, and external identifiers often need duplicate detection. This means a...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P11 Sets

Example and pitfalls

Find duplicate external IDs

The seen set makes duplicate detection direct and efficient. This pattern is useful before import jobs where duplicates would create confusing failures.

```
external_ids = ["sale.order_1", "sale.order_2", "sale.
seen = set()
duplicates = set()

for xml_id in external_ids:
    if xml_id in seen:
        duplicates.add(xml_id)
    seen.add(xml_id)

print(duplicates)
```

Common mistakes

- Expecting a set to preserve insertion order for presentation.
- Trying to put dictionaries or lists inside a set.
- Using {} when an empty set was intended.
- Converting to a set too early and losing meaningful duplicates.

P12 Conditionals: Truthy Values, Ternary Expressions, and Short-Circuiting

Objectives, key points, and lab target

Objectives

- Write if, elif, and else branches clearly.
- Use truthy values deliberately.
- Apply ternary expressions only when they improve readability.
- Explain and use short-circuit boolean evaluation.

Key points

- if starts a branch, elif adds alternatives, and else catches the remainder.
- and returns when the first falsy value is found; or returns when the first truthy value is...
- A ternary expression has the form a if condition else b.
- Short-circuiting can prevent unsafe access to missing values.
- Complex conditionals deserve named boolean variables.

Lab target

Write a decision function that returns an invoice approval state based on amount, customer status, and whether required fields are present. Refactor complex conditions into named booleans.

P12 Conditionals: Truthy Values, Ternary Expressions, and Short-Circuiting

Teaching notes

Conditionals: Truthy Values, Ternary Expressions, and Short-Circuiting is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding controlled branching based on conditions helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is Python evaluates conditions to a truth value and executes only the matching branch, while and and or may skip later expressions. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is compressing business rules until nobody can see which case is handled, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, computed fields, constraints, button actions, and controllers often branch by state, permissions, and...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P12 Conditionals: Truthy Values, Ternary Expressions, and Short-Circuiting

Example and pitfalls

Readable business branch

The branches read like a policy and make the exceptional cases visible. This is better than hiding the same rule in a dense expression.

```
state = "draft"
amount_total = 1500

if state != "draft":
    action = "readonly"
elif amount_total > 1000:
    action = "manager_approval"
else:
    action = "confirm"

# ...
```

Common mistakes

- Using truthiness when zero, False, and None have different meanings.
- Writing nested ternary expressions for business rules.
- Forgetting that and and or return operands, not always True or False.
- Placing expensive or unsafe checks before cheap guard checks.

Short-circuit guard

```
partner = {"name": "Azure", "email": ""}
message = partner.get("email") and f"Send mail to {partner
print(message or "No email available")
```

P13 for Loops, Iterables, range, and enumerate

Objectives, key points, and lab target

Objectives

- Iterate over sequences and other iterables.
- Use range for counted loops.
- Use enumerate when an index is genuinely needed.
- Write loop bodies that avoid accidental side effects.

Key points

- for item in items is the default readable loop.
- range produces integer sequences without storing a full list.
- enumerate provides index and value together.
- Loop variables are ordinary names and remain bound after the loop.
- Avoid modifying a list while iterating over it unless the pattern is deliberate.

Lab target

Validate a list of CSV-like row dictionaries. Use enumerate(start=1) to collect readable error messages that include line numbers.

P13 for Loops, Iterables, range, and enumerate

Teaching notes

for Loops, Iterables, range, and enumerate is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding repeated work over iterable values helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a for loop asks an iterable for values one at a time and binds each value to the loop variable. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is looping by index when direct iteration would be safer and clearer, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, Odoo recordsets, lists of values, import rows, and report lines are all iterated constantly. This means...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P13 for Loops, Iterables, range, and enumerate

Example and pitfalls

Enumerate import rows

enumerate keeps the human line number next to the row data without manual counters. This is ideal for import validation errors.

```
rows = [
    {"name": "A", "price": 10},
    {"name": "B", "price": 20},
]

for line_number, row in enumerate(rows, start=1):
    print(f"Line {line_number}: {row['name']} costs {row['price']}")
```

Common mistakes

- Using range(len(items)) when the item itself is all that is needed.
- Changing the list being iterated and skipping elements unexpectedly.
- Forgetting start=1 when reporting human line numbers.
- Using loop variables after the loop without considering empty input.

P14 while Loops, break, continue, and pass

Objectives, key points, and lab target

Objectives

- Use while loops for condition-driven repetition.
- Exit loops intentionally with break.
- Skip to the next iteration with continue.
- Use pass as an explicit placeholder only when appropriate.

Key points

- while is best when the number of iterations is not known upfront.
- break exits the nearest loop immediately.
- continue skips the rest of the current iteration.
- pass does nothing and should not hide missing implementation.
- Loop counters and retry limits prevent runaway scripts.

Lab target

Write a retry loop that attempts a fake operation up to three times. Use break on success, continue on a recoverable validation issue, and raise an error after all retries fail.

P14 while Loops, break, continue, and pass

Teaching notes

while Loops, break, continue, and pass is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding looping until a condition changes helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a while loop keeps executing while its condition remains truthy, so the body must move the program toward completion. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is creating infinite loops or hiding unfinished code behind pass, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, polling jobs, batch processing, and retry loops appear in integration and maintenance scripts around...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P14 while Loops, break, continue, and pass

Example and pitfalls

Batch loop with explicit progress

The loop condition is tied to the remaining work, and each iteration removes one item. continue skips one item without stopping the entire batch.

```
remaining = [1, 2, 3, 4, 5]
processed = []

while remaining:
    item = remaining.pop(0)
    if item == 3:
        continue
    processed.append(item)

print(processed)
```

Common mistakes

- Writing while True without a clear break condition.
- Using pass where a real implementation or explicit error should exist.
- Forgetting to update the condition variable.
- Catching all errors inside a loop and continuing forever.

P15 Functions: Parameters, args, kwargs, return, and Docstrings

Objectives, key points, and lab target

Objectives

- Define reusable functions with clear parameters.
- Use positional, keyword, *args, and **kwargs appropriately.
- Return values deliberately.
- Write docstrings that explain behavior and constraints.

Key points

- Parameters define what information a function needs.
- return sends a value back to the caller and stops function execution.
- *args collects extra positional arguments; **kwargs collects extra keyword arguments.
- Default arguments are evaluated once when the function is defined.
- Docstrings should state purpose, important parameters, return value, and notable errors.

Lab target

Write a function that accepts product price, quantity, and optional discount. Validate inputs, return a total, and include a docstring with one example call.

P15 Functions: Parameters, args, kwargs, return, and Docstrings

Teaching notes

Functions: Parameters, args, kwargs, return, and Docstrings is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding packaging behavior behind a callable interface helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a function receives arguments, binds them to parameter names, executes a body, and optionally returns a value. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is creating functions with vague contracts that callers misuse, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, Odoo model methods are functions with framework-provided self, decorators, context, and return...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P15 Functions: Parameters, args, kwargs, return, and Docstrings

Example and pitfalls

A clear calculation function

The signature exposes the required subtotal and optional discount, while validation protects the business rule. The docstring describes behavior rather than repeating the code.

```
def compute_discounted_total(subtotal, discount_rate=0
    """Return subtotal after applying a decimal discou
    if not 0 <= discount_rate <= 1:
        raise ValueError("discount_rate must be between 0 and 1")
    return subtotal * (1 - discount_rate)

print(compute_discounted_total(100, discount_rate=0.15
```

Common mistakes

- Using mutable default arguments such as [] or {}.
- Returning different types for similar outcomes without documenting them.
- Accepting **kwargs and silently ignoring misspelled options.
- Writing docstrings that say what is obvious instead of why constraints exist.

Forward keyword options

```
def log_action(action, **details):
    print(action)
    for key, value in sorted(details.items()):
        print(f"  {key}: {value}")

log_action("confirm_order", order="SO001", user="admin")
```

4 P16 – P20

Python for Odoo Developers · teaching block 4

P16 Scope, global, and nonlocal

Objectives, key points, and lab target

Objectives

- Explain local, enclosing, global, and built-in scopes.
- Understand name resolution with the LEGB rule.
- Use global and nonlocal sparingly and deliberately.
- Avoid hidden shared state in Odoo modules.

Key points

- LEGB means local, enclosing, global, built-in.
- Reading a global name is different from rebinding it.
- global tells Python assignment should target the module scope.
- nonlocal targets an enclosing function scope.
- Module constants are acceptable; mutable module caches need careful invalidation.

Lab target

Write a function that accidentally shadows a built-in, then fix the name. Next, create a small closure with nonlocal and explain why you would avoid the same pattern for request data.

P16 Scope, global, and nonlocal

Teaching notes

Scope, global, and nonlocal is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding where names are looked up and rebound helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is Python searches local, enclosing, global, and built-in scopes for a name, while assignment normally binds in the local scope. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is using hidden module-level state that behaves unpredictably under long-running workers, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, Odoo workers may serve many requests, so module globals can leak state between users or transactions....
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P16 Scope, global, and nonlocal

Example and pitfalls

Local and enclosing scope

nonlocal allows the inner function to rebind a name from the enclosing function. This is powerful, but in application code explicit objects are often easier to reason about.

```
def make_counter():
    count = 0

    def next_value():
        nonlocal count
        count += 1
        return count

    return next_value

# ...
```

Common mistakes

- Using global to avoid passing parameters.
- Storing request-specific data in module-level variables.
- Shadowing built-in names such as list, dict, or id.
- Assuming each web request starts with fresh module globals.

P17 OOP: Classes, __init__, and Methods

Objectives, key points, and lab target

Objectives

- Define classes and create instances.
- Use __init__ to initialize object state.
- Write instance methods that use self correctly.
- Connect Python classes to Odoo models.

Key points

- class creates a new type.
- __init__ runs when an instance is created.
- self is the current instance passed automatically to instance methods.
- Instance attributes belong to one object unless shared deliberately.
- Methods should operate on the object's coherent state.

Lab target

Create a plain ProductLabel class with name, default_code, and price. Add a method that returns a formatted label and test it with two instances.

P17 OOP: Classes, __init__, and Methods

Teaching notes

OOP: Classes, __init__, and Methods is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding bundling data and behavior into objects helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a class defines behavior, an instance holds state, and methods receive the instance as self. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is placing behavior in disconnected helper code when object state would make the design clearer, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, Odoo models are Python classes whose fields and methods are extended by the framework metaclass and...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P17 OOP: Classes, __init__, and Methods

Example and pitfalls

Plain class with state

The instance owns number and total, and the method formats state through self. This mirrors how model methods organize behavior around record data.

```
class InvoiceSummary:
    def __init__(self, number, total):
        self.number = number
        self.total = total

    def label(self):
        return f"{self.number}: {self.total:.2f}"

summary = InvoiceSummary("INV001", 250.0)
print(summary.label())
```

Common mistakes

- Forgetting self in method definitions.
- Using class attributes when instance attributes were intended.
- Doing too much work in __init__ instead of keeping construction simple.
- Assuming an Odoo recordset method always receives a single record.

P18 classmethod and staticmethod

Objectives, key points, and lab target

Objectives

- Distinguish instance methods, class methods, and static methods.
- Use classmethod for alternate constructors or class-level behavior.
- Use staticmethod only when no instance or class state is needed.
- Avoid unnecessary method types in framework code.

Key points

- Instance methods are the default and receive self.
- classmethod receives the class and can create instances polymorphically.
- staticmethod is namespaced on the class but has no automatic access to state.
- Alternate constructors are a common classmethod use case.
- Utility functions do not always need to live inside a class.

Lab target

Create a small CodeParser class with a classmethod from_text and a staticmethod is_valid_code. Explain which method would become an instance method if it needed stored state.

P18 classmethod and staticmethod

Teaching notes

classmethod and staticmethod is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding choosing the right method binding helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is instance methods receive self, class methods receive cls, and static methods receive neither automatically. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is using method decorators to hide unclear responsibilities, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, Odoo has its own API decorators, so plain Python method decorators should be used only when the...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P18 classmethod and staticmethod

Example and pitfalls

Alternate constructor

from_string receives cls, so subclasses could reuse the constructor correctly. The currency check is static because it does not need instance or class state.

```
class Money:
    def __init__(self, amount, currency):
        self.amount = amount
        self.currency = currency

    @classmethod
    def from_string(cls, text):
        amount_text, currency = text.split()
        return cls(float(amount_text), currency)

# ...
```

Common mistakes

- Using staticmethod to avoid thinking about where a function belongs.
- Using classmethod when no class-level behavior exists.
- Confusing Python decorators with Odoo api decorators.
- Putting business logic in utility methods that bypass record rules or environment context.

P19 Encapsulation, Inheritance, Polymorphism, and super

Objectives, key points, and lab target

Objectives

- Explain encapsulation as controlled access to behavior and state.
- Use inheritance to specialize behavior.
- Recognize polymorphism through shared method contracts.
- Call parent behavior correctly with super.

Key points

- Encapsulation keeps callers focused on behavior rather than internal representation.
- Inheritance models an is-a relationship or framework extension point.
- Polymorphism lets different classes respond to the same method call.
- super delegates to the next implementation in the method resolution order.
- Overridden methods must preserve expected arguments, return values, and side effects.

Lab target

Build a base Approval class and two subclasses with different validation rules. Ensure each subclass calls super and returns the same type.

P19 Encapsulation, Inheritance, Polymorphism, and super

Teaching notes

Encapsulation, Inheritance, Polymorphism, and super is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding object-oriented extension patterns helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is objects expose behavior through methods, subclasses can override methods, and callers can rely on shared method names. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is overriding framework methods without preserving parent behavior or contracts, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, Odoo customization frequently inherits models and extends methods such as create, write, unlink, and...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P19 Encapsulation, Inheritance, Polymorphism, and super

Example and pitfalls

Extend behavior with super

The subclass adds invoice-specific behavior without discarding the parent checks. This is the same mindset needed when extending Odoo model methods.

```
class Document:
    def validate(self):
        return ["base checks"]

class Invoice(Document):
    def validate(self):
        checks = super().validate()
        checks.append("invoice checks")
        return checks

# ...
```

Common mistakes

- Overriding a method and forgetting super when parent behavior is required.
- Changing the return type of an inherited method.
- Using inheritance where composition or a helper function would be clearer.
- Accessing internal attributes from outside instead of using methods.

P20 Dunders, MRO, and Multiple Inheritance

Objectives, key points, and lab target

Objectives

- Recognize common double-underscore methods.
- Explain method resolution order.
- Understand the risks and uses of multiple inheritance.
- Design cooperative methods that work with super.

Key points

- __repr__, __str__, __len__, and __bool__ customize common Python operations.
- MRO can be inspected with ClassName.mro().
- super follows MRO, not simply the textual parent class.
- Multiple inheritance works best when methods accept compatible signatures.
- Mixins should be small and focused.

Lab target

Create two mixins and a base class with a cooperative process method. Print the MRO and show the order in which each method runs.

P20 Dunders, MRO, and Multiple Inheritance

Teaching notes

Dunders, MRO, and Multiple Inheritance is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding Python's data model and inheritance lookup helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is dunder methods let objects participate in Python protocols, and MRO defines the order used to find methods. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is creating inheritance diamonds that break because methods do not cooperate, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, Odoo models combine Python inheritance with framework inheritance mechanisms, so method lookup must be...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P20 Dunders, MRO, and Multiple Inheritance

Example and pitfalls

Inspect MRO

The MRO shows the actual lookup order, and super follows that order cooperatively. This is why mixins should call super when they participate in a shared method chain.

```
class AuditMixin:
    def save(self):
        print("audit")
        return super().save()

class Base:
    def save(self):
        print("base")
        return True

# ...
```

Common mistakes

- Assuming super always calls the immediate parent class.
- Writing mixins that do not call super in cooperative chains.
- Using dunder methods for cleverness instead of protocol clarity.
- Creating multiple inheritance hierarchies with incompatible method signatures.

Readable object representation

```
class PartnerRef:
    def __init__(self, name, ref):
        self.name = name
        self.ref = ref

    def __repr__(self):
        return f"PartnerRef(name={self.name!r}, ref={self.
# ...
```

5 P21 – P25

Python for Odoo Developers · teaching block 5

P21 map, filter, zip, reduce, and lambdas

Objectives, key points, and lab target

Objectives

- Use functional helpers where they improve clarity.
- Write simple lambda expressions.
- Combine related iterables with zip.
- Recognize when comprehensions or explicit loops are clearer.

Key points

- map applies a function to each item.
- filter keeps items whose predicate is truthy.
- zip pairs items from iterables by position.
- functools.reduce combines items into one result but can be less readable than a loop.
- lambda creates a small anonymous function expression.

Lab target

Transform a list of product dictionaries into display names using both map and a comprehension. Explain which version your team should keep and why.

P21 map, filter, zip, reduce, and lambdas

Teaching notes

map, filter, zip, reduce, and lambdas is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding functional iteration tools helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is functions can be passed around as values, and iterable helpers apply them lazily or combine iterable streams. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is choosing clever expressions that hide business logic and complicate debugging, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, Odoo code benefits from readable transformations of records and values, but framework recordsets already...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P21 map, filter, zip, reduce, and lambdas

Example and pitfalls

Transform and pair values

map performs a simple transformation, while zip pairs two related sequences. Converting map to a list is necessary when you want to inspect all results immediately.

```
names = ["sale", "crm", "stock"]
labels = list(map(str.upper, names))
enabled = [True, False, True]

for module, is_enabled in zip(names, enabled):
    print(module, is_enabled)

print(labels)
```

Common mistakes

- Using lambda for complex logic that deserves a named function.
- Forgetting that map and filter are lazy iterators in Python 3.
- Using zip when input lengths must be validated and may differ.
- Using reduce where sum, any, all, or a loop is clearer.

Reduce with care

```
from functools import reduce

amounts = [10, 20, 30]
total = reduce(lambda current, amount: current + amount, a
print(total)
```

P22 Comprehensions

Objectives, key points, and lab target

Objectives

- Write list, dict, and set comprehensions.
- Use conditional clauses in comprehensions responsibly.
- Replace simple loops with clear comprehensions.
- Avoid dense comprehensions for business-heavy logic.

Key points

- List comprehensions build lists.
- Set comprehensions build unique sets.
- Dict comprehensions build key-value mappings.
- Filters appear after the for clause with if.
- Nested comprehensions should be used sparingly.

Lab target

Given a list of order dictionaries, create a list of confirmed order names, a set of customer names, and a dictionary mapping order name to total.

P22 Comprehensions

Teaching notes

Comprehensions is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding concise collection construction helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a comprehension builds a new collection by iterating over an input and optionally filtering or transforming items. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is packing too much logic into one line and making errors hard to locate, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, Odoo developers often build domains, value lists, summaries, and validation messages from records or...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P22 Comprehensions

Example and pitfalls

Build validation messages

The comprehension cleanly maps invalid rows into error messages. If validation had multiple rules and side effects, a normal loop would be easier to maintain.

```
rows = [
    {"line": 1, "name": "A", "price": 10},
    {"line": 2, "name": "", "price": 5},
]

errors = [f"Line {row['line']}: missing name" for row in rows if not row["name"]]
print(errors)
```

Common mistakes

- Using comprehensions for print, write, or other side effects.
- Writing nested comprehensions that require careful mental parsing.
- Forgetting duplicate keys overwrite earlier values in dict comprehensions.
- Hiding validation rules inside a long conditional expression.

Create a lookup dictionary

```
products = [
    {"default_code": "A001", "name": "Desk"},
    {"default_code": "A002", "name": "Chair"},
]

by_code = {product["default_code"]: product for product in
print(by_code["A001"]["name"])
```

P23 Decorators

Objectives, key points, and lab target

Objectives

- Explain decorators as functions that wrap or modify callables.
- Write a simple decorator with functools.wraps.
- Understand why decorators affect call behavior.
- Relate Python decorators to Odoo api decorators carefully.

Key points

- The @decorator syntax is assignment sugar around function wrapping.
- functools.wraps preserves name, docstring, and metadata.
- Decorators can add logging, validation, caching, or framework registration.
- The wrapper should accept compatible arguments.
- Odoo decorators are framework contracts, not merely style markers.

Lab target

Write a decorator that logs function arguments and return value for a pure calculation function. Then explain why the same decorator might be unsafe around sensitive production data.

P23 Decorators

Teaching notes

Decorators is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding callable wrapping and metadata-preserving behavior helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a decorator receives a function and returns a replacement callable, often adding work before or after the original call. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is using decorators without understanding that they change the callable being executed, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, Odoo uses decorators such as api.depends, api.constrains, and api.model to tell the framework how...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P23 Decorators

Example and pitfalls

Simple timing decorator

The decorator adds timing around the original function while returning its result. wraps keeps debugging and documentation output tied to the original function name.

```
from functools import wraps
from time import perf_counter

def timed(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
# ...
```

Common mistakes

- Forgetting to return the original function's result from the wrapper.
- Not using functools.wraps and losing useful metadata.
- Changing arguments in a wrapper without a clear contract.
- Copying Odoo decorators without understanding what the ORM expects.

P24 Error Handling

Objectives, key points, and lab target

Objectives

- Raise and catch exceptions intentionally.
- Use try, except, else, and finally appropriately.
- Design error messages for operators and users.
- Avoid swallowing framework errors.

Key points

- raise creates an exception.
- except handles matching exception types.
- else runs when no exception occurred.
- finally runs whether success or failure occurred.
- Specific exception types are safer than bare except.

Lab target

Write a parser for imported quantities that reports line numbers on invalid input. Catch only expected conversion errors and collect all row errors for display.

P24 Error Handling

Teaching notes

Error Handling is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding explicit handling of exceptional control flow helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is exceptions interrupt normal execution and travel up the call stack until code handles them or the program reports failure. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is catching broad exceptions and hiding the actual cause of data or transaction problems, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, Odoo translates many exceptions into user-visible errors and rolls back failed transactions. This means...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P24 Error Handling

Example and pitfalls

Validate input with a specific exception

The function catches only conversion errors and raises a clearer domain message while preserving the original cause. Validation after conversion keeps separate failure modes...

```
def parse_positive_int(text):
    try:
        value = int(text)
    except ValueError as exc:
        raise ValueError(f"expected a whole number, got {text!r}") from exc
    if value <= 0:
        raise ValueError("value must be positive")
    return value

print(parse_positive_int("12"))
```

Common mistakes

- Using bare except and hiding KeyboardInterrupt or system exit.
- Catching an error only to print it and continue with corrupt state.
- Raising generic Exception when a specific type would be clearer.
- Discarding the original exception cause during re-raise.

Always close local resources

```
path = "/tmp/python-error-lab.txt"
handle = open(path, "w")
try:
    handle.write("hello\n")
finally:
    handle.close()
```

P25 Generators

Objectives, key points, and lab target

Objectives

- Define generator functions with yield.
- Explain lazy iteration and one-pass consumption.
- Use generators for streaming large data.
- Avoid generator surprises in debugging and reuse.

Key points

- yield turns a function into a generator function.
- Generators are iterators and are consumed as they are iterated.
- Lazy evaluation can reduce memory use.
- list(generator) materializes all values and consumes the generator.
- Generator expressions are concise lazy expressions.

Lab target

Write a generator that yields valid invoice rows from raw dictionaries. Demonstrate that iterating it twice without recreating it produces values only the first time.

P25 Generators

Teaching notes

Generators is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding lazy production of values helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a generator function pauses at yield and resumes later, producing values only as the caller requests them. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is assuming a generator is a reusable list and accidentally consuming it once, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, large exports, imports, and reports can benefit from streaming rather than loading everything into...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P25 Generators

Example and pitfalls

Stream cleaned rows

The generator yields only valid cleaned rows and skips empty names without building an intermediate list. This pattern scales well for import pipelines.

```
def cleaned_rows(raw_rows):
    for row in raw_rows:
        name = row.get("name", "").strip()
        if not name:
            continue
        yield {"name": name, "price": float(row.get("price", 0))}

rows = [{"name": " Desk ", "price": "10.5"}, {"name": "", "price": "2"}]
for row in cleaned_rows(rows):
    print(row)
```

Common mistakes

- Iterating a generator once and expecting values to remain for a second pass.
- Materializing a huge generator too early with list.
- Putting hidden side effects inside a generator and forgetting when they execute.
- Returning a list when the caller expects lazy behavior.

6 P26 – P28

Python for Odoo Developers · teaching block 6

P26 Modules, Packages, Imports, and __name__

Objectives, key points, and lab target

Objectives

- Organize code into modules and packages.
- Use import forms responsibly.
- Explain how Python finds modules.
- Use __name__ == '__main__' for script entry points.

Key points

- A .py file is a module; a package is a directory Python can import.
- import module keeps names qualified; from module import name brings a name directly into scope.
- Python searches sys.path to find modules.
- Module top-level code runs at import time.
- __name__ is '__main__' when a file is run as a script.

Lab target

Create a two-file package with a helper module and a script module. Import the helper from the script and protect executable behavior with a main guard.

P26 Modules, Packages, Imports, and __name__

Teaching notes

Modules, Packages, Imports, and __name__ is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding code organization and import execution helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a module is executed once when imported, names become attributes, and packages group modules with importable paths. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is putting side effects at import time and surprising tests, scripts, or Odoo module loading, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, Odoo addons are Python packages whose __init__.py files import models, controllers, and other framework...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P26 Modules, Packages, Imports, and __name__

Example and pitfalls

Use a main guard

The main guard prevents script behavior from running when the module is imported elsewhere. This is essential for reusable utilities and safe tests.

```
def main():
    print("run as a script")

if __name__ == "__main__":
    main()
```

Common mistakes

- Executing database or network work at import time.
- Using wildcard imports and hiding where names come from.
- Creating circular imports between modules.
- Forgetting to import a new Odoo models file in the package __init__.py.

Prefer qualified imports for clarity

```
import pathlib

config_path = pathlib.Path("/etc/odoo/odoo.conf")
print(config_path.exists())
```

P27 pip, venv, and PyPI

Objectives, key points, and lab target

Objectives

- Create and activate a Python virtual environment.
- Install packages with pip into the intended environment.
- Explain PyPI as a package index.
- Manage dependencies safely for Odoo projects.

Key points

- venv creates an isolated environment directory.
- Activating a venv changes PATH so python and pip point into that environment.
- python -m pip ensures pip belongs to the selected interpreter.
- PyPI hosts public Python packages, but packages still require review.
- requirements files should be pinned or constrained for production.

Lab target

Create a virtual environment, install one small package, print the interpreter path, freeze dependencies, deactivate, and explain why the package is not necessarily available to system Python.

P27 pip, venv, and PyPI

Teaching notes

pip, venv, and PyPI is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding isolated dependency management helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is a virtual environment provides a separate interpreter context with its own installed packages. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is installing dependencies into the wrong environment or accepting unreviewed packages, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, Odoo deployments depend on exact Python packages for Odoo itself, custom addons, integrations, and...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P27 pip, venv, and PyPI

Example and pitfalls

Create and inspect a virtual environment

Using python -m pip ties package installation to the currently selected interpreter. Printing sys.executable confirms the shell is using the virtual environment.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -c "import sys; print(sys.executable)"
python -m pip list
```

Common mistakes

- Running pip without checking which Python it targets.
- Installing project packages into the system Python on a production host.
- Depending on unpinned package versions for repeatable deployments.
- Adding packages without license, maintenance, or security review.

Install from a requirements file

```
cat > requirements.txt <<'REQ'
requests
REQ

python -m pip install -r requirements.txt
python -m pip show requests
```

P28 Debugging and Python Habits for Odoo Developers

Objectives, key points, and lab target

Objectives

- Apply a disciplined debugging workflow.
- Use print, logging, pdb, and tests appropriately.
- Develop Python habits that fit Odoo's ORM and transaction model.
- Turn fixes into maintainable changes.

Key points

- Start with the exact error, input, user, company, and database state.
- Use logging for server code where print may be hidden or noisy.
- pdb is useful locally but dangerous if left in server code.
- Respect recordsets: methods may receive zero, one, or many records.
- Do not bypass the ORM unless you understand transactions, security, and cache invalidation.

Lab target

Take a failing Python function, write down the expected input and output, add two focused assertions, fix the function, then describe how the same habit applies before changing an Odoo model method.

P28 Debugging and Python Habits for Odoo Developers

Teaching notes

Debugging and Python Habits for Odoo Developers is important because Python is both the language of Odoo customization and the daily tool for automation around Odoo. Understanding evidence-driven debugging and framework-aware habits helps developers read existing modules, change behavior deliberately, and avoid accidental side effects.

The useful mental model is debugging is a loop of observing, forming a hypothesis, making a minimal test, and verifying the result. When that model is clear, syntax stops being a guessing game and becomes a way to express intent precisely.

Python is designed for readability, but readable code still needs disciplined structure. The main risk in this lesson is changing code until the symptom disappears without understanding the cause, especially when code runs inside a large framework with many inherited conventions.

Also remember

- In Odoo work, Odoo code runs inside transactions, environments, caches, access rules, and long-running worker...
- Senior developers do not stop at making an example run once. They ask what type each value has, who owns the data,...

P28 Debugging and Python Habits for Odoo Developers

Example and pitfalls

Use logging instead of print in server code

Logging keeps diagnostic output integrated with server log configuration and preserves structured context. The message uses lazy formatting so values are formatted only when the...

```
import logging

_logger = logging.getLogger(__name__)

def confirm_order(order_name, amount_total):
    _logger.info("confirming order %s amount=%s", order_name, amount_total)
    if amount_total <= 0:
        raise ValueError("amount_total must be positiv
    return True
```

Common mistakes

- Leaving breakpoint calls in code that may run on a server.
- Debugging as administrator only and missing access-rule failures.
- Calling ensure_one reflexively instead of designing for recordsets.
- Fixing a symptom without adding a regression test or prevention note.

Small reproduction before framework change

```
def normalize_code(value):
    return value.strip().upper().replace(" ", "_")

assert normalize_code(" sale order ") == "SALE_ORDER"
assert normalize_code("crm") == "CRM"
print("normalization rules understood")
```

P28 Debugging and Python Habits for Odoo Developers

Extra patterns

Odoo Python habits

- Use the ORM for business data unless there is a reviewed reason not...
- Treat recordsets as collections until a method explicitly requires...
- Keep imports cheap and module loading predictable.
- Log with context and avoid leaking secrets.

Σ You should now be able to...

Section wrap-up

- P01 What Is a Programming Language and Interpreter
- P02 Running Python, First Program, and Python 2 vs Python 3
- P03 Data Types Overview
- P04 Numbers, Math, and Operator Precedence
- P05 Variables, Expressions, and Statements
- P06 Strings: Concatenation, Formatting, Indexes, and Immutability
- P07 Booleans and Type Conversion
- P08 Lists: Slicing, Matrix Data, Methods, and Unpacking

- Complete every lab before the next section
- Keep a command/error journal
- Prefer disposable databases for experiments
- Re-read pitfalls before production changes

Python 3 | OOP | venv | PEP 8

NEXT STEP

Complete the section labs

Open the next presentation deck

Section 02 · Python